

# Http , Https,

🕒 Created July 20, 2026 10:12 PM

## HTTP, HTTPS, Cookies & Sessions — Quick Notes

### HTTP (Hypertext Transfer Protocol)

| Property   | Value                        |
|------------|------------------------------|
| Port       | 80                           |
| Encryption | ❌ None — plaintext           |
| Model      | Request-response (stateless) |

### Key HTTP Methods

| Method  | Use          | Safe? | Note                              |
|---------|--------------|-------|-----------------------------------|
| GET     | Retrieve     | ✅     | Params in URL — logged everywhere |
| POST    | Create       | ❌     | Body data — more secure than GET  |
| PUT     | Replace      | ❌     | Test for mass assignment          |
| DELETE  | Delete       | ❌     | Critical ACL check                |
| OPTIONS | List methods | ✅     | Reveals attack surface            |
| TRACE   | Echo request | ✅     | <b>DISABLE</b> — XST attacks      |

### Important Status Codes

| Code    | Meaning   | Check                                         |
|---------|-----------|-----------------------------------------------|
| 200     | OK        | Was auth enforced?                            |
| 301/302 | Redirect  | Open redirect?                                |
| 403     | Forbidden | ✅ Correct                                     |
| 404     | Not Found | Some apps return 404 instead of 403 — bypass? |

| Code | Meaning | Check            |
|------|---------|------------------|
| 500  | Error   | Info disclosure? |

## HTTPS (HTTP + TLS)

| Property   | Value               |
|------------|---------------------|
| Port       | 443                 |
| Encryption | ✓ TLS 1.2 / 1.3     |
| Auth       | ✓ X.509 certificate |

### What HTTPS Protects

- **Confidentiality** — data is encrypted in transit
- **Integrity** — data can't be tampered with
- **Authentication** — server identity is verified by a CA

### Attacks on HTTPS

| Attack        | How                 | Fix              |
|---------------|---------------------|------------------|
| SSL Stripping | Downgrade to HTTP   | HSTS header      |
| Weak Ciphers  | Old TLS 1.0/1.1     | Enforce TLS 1.2+ |
| Heartbleed    | OpenSSL memory leak | Patch OpenSSL    |
| Rogue CA      | Fake cert accepted  | Check cert chain |

### HSTS Header

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

- Tells the browser: **HTTPS only** for N seconds
- Test: `curl -sI https://target.com | grep -i strict-transport-security`

### Security Headers Checklist

| Header                                 | Purpose       | Value                                            |
|----------------------------------------|---------------|--------------------------------------------------|
| <code>Strict-Transport-Security</code> | Enforce HTTPS | <code>max-age=63072000; includeSubDomains</code> |

| Header                               | Purpose                   | Value                                        |
|--------------------------------------|---------------------------|----------------------------------------------|
| <code>Content-Security-Policy</code> | Restrict loaded resources | <code>default-src 'self'</code>              |
| <code>X-Content-Type-Options</code>  | No MIME sniffing          | <code>nosniff</code>                         |
| <code>X-Frame-Options</code>         | Prevent clickjacking      | <code>DENY</code>                            |
| <code>Referrer-Policy</code>         | Control referrer leakage  | <code>strict-origin-when-cross-origin</code> |

## Cookies (Client-Side State)

### Cookie Attributes

| Attribute                                   | Purpose                                    | Must Have?                  |
|---------------------------------------------|--------------------------------------------|-----------------------------|
| <code>HttpOnly</code>                       | Blocks <code>document.cookie</code> access | ✓ YES — stops XSS theft     |
| <code>Secure</code>                         | Only sent over HTTPS                       | ✓ YES — stops MitM          |
| <code>SameSite</code>                       | Controls cross-origin sending              | ✓ YES — prevents CSRF       |
| <code>Domain</code>                         | Which domains receive the cookie           | ⚠ Keep narrow               |
| <code>Path</code>                           | Limits to a specific path                  | ⚠ Minimal value             |
| <code>Expires</code> / <code>Max-Age</code> | Lifetime                                   | ✓ Session cookies preferred |

### SameSite Values

| Value               | Behavior                                        | CSRF Protection           |
|---------------------|-------------------------------------------------|---------------------------|
| <code>Strict</code> | No cross-origin sends                           | ✓ Strongest               |
| <code>Lax</code>    | Top-level navigations only (default since 2021) | ✓ Good                    |
| <code>None</code>   | All cross-origin (must be <code>Secure</code> ) | ✗ None — needs CSRF token |

### Cookie Attacks

| Attack            | Description                           | Fix                                                                 |
|-------------------|---------------------------------------|---------------------------------------------------------------------|
| Session Hijacking | Steal cookie → impersonate user       | <code>HttpOnly</code> + <code>Secure</code> + <code>SameSite</code> |
| Session Fixation  | Force a known session ID before login | Regenerate ID on login                                              |

| Attack         | Description                             | Fix                                         |
|----------------|-----------------------------------------|---------------------------------------------|
| Cookie Tossing | Subdomain overrides parent cookie       | <code>__Host-</code> prefix                 |
| CSRF           | Cookie auto-attaches to forged requests | <code>SameSite=Strict</code> or CSRF tokens |

## Sessions (Server-Side State)

### What a Session Does

- Stores user data (user\_id, role) **on the server**
- Identified by a **session ID** stored in a cookie
- Solves HTTP's stateless problem

### Session Lifecycle

```

Anonymous → Server creates session
↓
Login → Regenerate session ID (CRITICAL)
↓
Authenticated → Session stores user_id, role
↓
Logout / Timeout → Destroy session (server + cookie)

```

### Session ID Requirements

| Requirement   | Why                     | Standard             |
|---------------|-------------------------|----------------------|
| Unpredictable | Prevent guessing        | CSPRNG, ≥ 128 bits   |
| Unique        | No collisions           | 32+ hex chars        |
| Expiring      | Limit compromise window | 15–30 min inactivity |
| Not in URL    | Prevent leakage         | Cookie only          |

### Session Attacks

| Attack            | Description         | Fix                      |
|-------------------|---------------------|--------------------------|
| Session Hijacking | Steal session ID    | Secure + HttpOnly cookie |
| Session Fixation  | Set ID before login | Regenerate on login      |

| Attack              | Description                    | Fix                     |
|---------------------|--------------------------------|-------------------------|
| Session Prediction  | Weak generation                | CSPRNG + long entropy   |
| Session Sidejacking | Sniff over WiFi                | Secure flag + HTTPS     |
| No Invalidation     | Logout doesn't destroy session | Destroy server + cookie |
| No Regeneration     | Same ID pre/post login         | Always regenerate       |

## Testing Commands (Quick Reference)

bash

```
# HTTP → HTTPS redirect?
curl -sI http://target.com

# Check TLS & ciphers
nmap --script ssl-enum-ciphers -p 443 target.com
ssllscan target.com

# Check certificate
echo |openssl s_client -connect target.com:443 -servername target.com 2>/dev/null |openssl x509 -text

# Check security headers
curl -sI https://target.com |grep -iE 'strict-transport|csp|x-frame|x-content'

# Check cookies
curl -sI https://target.com |grep -i set-cookie

# Check SameSite (none = vulnerable)
curl -sI https://target.com |grep -i samesite

# Test TLS version
openssl s_client -connect target.com:443 -tls1 # should fail
openssl s_client -connect target.com:443 -tls1_2 # should work
```

## Golden Rules

| # | Rule                                                              |
|---|-------------------------------------------------------------------|
| 1 | HTTPS everywhere — redirect HTTP → HTTPS                          |
| 2 | <b>HttpOnly</b> + <b>Secure</b> + <b>SameSite</b> on every cookie |
| 3 | Regenerate session ID on login & privilege escalation             |
| 4 | Destroy session on logout (server + cookie)                       |

| #  | Rule                                                           |
|----|----------------------------------------------------------------|
| 5  | Use CSPRNG — $\geq$ 128-bit session IDs                        |
| 6  | Set inactivity timeout (30 min) + absolute timeout (24h)       |
| 7  | Never put session ID in URL                                    |
| 8  | Invalidate all sessions on password change                     |
| 9  | Disable TRACE method — prevent XST                             |
| 10 | Don't trust the client for authorization — enforce server-side |